AGE SYSTEM 5.0.7

Active Effects Handbook

Conditions, stat paths, stacking, automation, examples, and troubleshooting for Foundry VTT 14

ICON / CONDITION	ACTIVE EFFECT	PREPARED ACTOR	VISIBLE RESULT
A token status, Character Effect, or automation action	One or more typed changes	Foundry applies the changes without rewriting the stored base value	Rolls and sheet totals reflect the temporary result

Scope of this handbook

This document describes the exact retained AGE System 5.0.7 automation build currently installed and the later Foundry 14.366 compatibility fixes carried by that retained copy. It does not describe the lean GitHub v14-foundation branch or the creator's separate official 5.0.0 release.

Prepared for GMs, players, module authors, and macro writers

1 Start here

An Active Effect is a temporary overlay on an Actor. It can change a statistic while enabled and immediately stop changing it when disabled or removed.

The three related tools

Tool	Best use	Where it lives
Token condition	A named state shown as a token icon, such as Hindered or Injured. Some conditions include mechanical changes; others are reminders only.	Token HUD or the Character Effects tab
Actor Active Effect	A custom temporary bonus or penalty attached to one Actor.	Full Character sheet - Effects - Custom
Item Modifier	A bonus supplied by an equipped/activated Item, including Focus, Power Force, item damage, or item activation bonuses.	The Item's Modifiers section

Do not confuse Item Modifiers with Active Effects

The Item Modifier types Power Force, Focus, Item Damage, Item Activation, All Power Force, and Focus Power Force have no direct Active Effect stat path. Configure those on the Item. Active Effects are best for Actor-level totals, abilities, defenses, armor, movement, maneuvers, and maximum resources.

Quick decision guide

- Need an icon and a reusable world condition? Use the GM Conditions Workshop.
- Need one character to have a temporary blessing, curse, injury, or stance? Create an Actor Active Effect.
- Need a sword, armor, talent, or Power to grant a bonus only while equipped or active? Use an Item Modifier.
- Need to spend or restore current Health, Power Points, or Conviction? Use AGE Actions or the sheet controls, not an Active Effect.

Contents

Section	Topic
2	How v14 applies a change
3	Built-in conditions
4	Conditions Workshop
5	Per-Actor effects
6	Stat path reference
7	Ability paths
8	Ready-made examples
9	Automation and timed effects
10	Troubleshooting
11	One-page cheat sheet
A	Advanced API examples and source references
B	Complete safe Active Effect key catalog

2 How Foundry v14 applies a change

Every change has four important fields

Field	Meaning	Example
Key	The exact AGE data path to affect. Always use system.* in Foundry v14.	`system.defense.total`
Type	How the effect combines its value with the existing value.	add
Value	The amount or replacement value. Numeric effects should use a numeric string.	2
Phase	initial for source/input fields; final for AGE totals calculated during preparation.	final

Change types

Type	Use it for	Example
add	Bonuses and penalties. This is the safest default for numeric AGE statistics.	+2 Defense or -1 tests
multiply	Halving or doubling a numeric result.	0.5 Speed
override	Forcing one exact value while the effect is active.	Speed becomes 0
upgrade	Keep the higher of the current and supplied value.	Minimum temporary floor
downgrade	Keep the lower of the current and supplied value.	Maximum temporary ceiling
custom	Only when system/module code explicitly interprets the value. Avoid for ordinary AGE stats.	Advanced integrations only

Initial versus final

- Use final for totals that AGE calculates: ability totals, Defense, Speed, armor totals, maximum resources, test/attack/damage modifiers, initiative, and similar derived values.
- Use initial for input fields that AGE reads while calculating later totals, such as an ability's testMod or a maneuver's defenseBonus.
- The Conditions Workshop automatically assigns final when you type a recognized final-phase key. Automation-created effects are normalized the same way.
- For a manually created effect in Foundry's native sheet, verify the phase yourself.

> **The effect does not permanently edit the Actor**
>
> Foundry applies Active Effects to prepared data. Disabling or deleting the effect reveals the stored base statistic again. This is why Active Effects are appropriate for temporary states and inappropriate for spending a resource.

3 Built-in condition set

The retained build starts with the AGE/Expanse condition set plus familiar Foundry status icons and numeric markers 0-10. Most conditions communicate a rule state; only the conditions listed below include automatic stat changes in the shipped data.

Condition	Mechanical change	Notes
Exhausted	Speed x 0.5	Final phase; also communicates restrictions described by the condition.
Hindered	Speed x 0.5	Final phase.
Injured	All tests -1	Adds -1 to system.testMod at final phase.
Wounded	Speed x 0.5; all tests -2	Two final-phase changes.
Restrained	Speed = 0	Final override.
Unconscious	Speed = 0	Final override.
Dying	No direct stat change	AGE automation can prompt at Health 0 and track dying rounds.

Reminder-only AGE conditions

Blinded, Deafened, Fatigued, Free Falling, Helpless, Prone, and the descriptive parts of other conditions do not automatically adjudicate every rule. Their icons and descriptions remind the table what must be handled manually.

Other available markers

The palette also carries Dead, number markers, Asleep, Flying, Stunned, Paralysis, Silenced, Fear, Burning, Frozen, Shocked, Corroding, Bleeding, Disease, Poisoned, Cursed, Regeneration, Degeneration, Invisible, Targeted, Blessed, Holy Shield, and related Foundry icons. Unless a condition definition contains a change, the icon alone does not modify an AGE statistic.

> **A condition can be descriptive and still be useful**
>
> Do not assume that a visible status icon has changed the dice calculation. Hover the condition on the Character Effects tab: the tooltip lists its description and any attached changes.

Dying assistance

- When Character Health reaches 0, the system can post a chat prompt to apply Dying.
- When Health rises above 0, an active Dying effect is disabled.
- At turn start, GM-side automation records the number of dying rounds and posts a reminder.
- The system deliberately does not auto-kill a character.

4 Build reusable conditions in the GM Conditions Workshop

The Conditions Workshop edits the world-level token condition palette. It is GM-only and is opened from Canvas Controls - AGE Toolbox - Conditions Workshop.

Step	What to do	Why
1	Open Conditions Workshop and select Custom.	The shipped AGE/Expanse set is read-only.
2	Choose Copy AGE/Expanse Effects, or Add Effect at top/bottom.	Copying is the easiest safe starting point.
3	Enter a unique name, description, and image.	The workshop rejects repeated icons in the custom set.
4	Enable Is Condition if it should appear in the Character condition list.	A custom Active Effect and a token condition are related but not identical concepts.
5	Add one or more changes: key, type, and value.	Recognized derived keys receive final phase automatically.
6	Save & Close, then test on a disposable Actor.	The custom set becomes the world token palette immediately.

Safe first custom condition

Field	Value
Name	Guarded
Is Condition	Yes
Change key	`system.defense.total`
Type	add
Value	2
Expected phase	final

Apply Guarded to a test Actor. Confirm that Defense increases by exactly 2, reload the world, confirm it remains, then disable it and confirm the original Defense returns.

Backup and portability

- Use Export in the Workshop to save the custom condition set as JSON.
- Use Import to load a saved custom set. Imported legacy icon/mode fields are normalized for v14.
- The Dead entry is always retained and cannot be removed from the Workshop.
- Keep an exported JSON copy before reorganizing a large condition library.

5 Create an effect for one Actor

Use this workflow for a temporary blessing, curse, spell, circumstance, injury, environmental modifier, or special scene state that should not become a reusable token condition.

Full Character sheet workflow

- Open the full Character sheet and select Effects.
- In Custom, choose the plus button. AGE creates a disabled one-round effect and opens Foundry's native v14 Active Effect sheet.
- Give it a clear name and image. In Changes, add the exact key, type, value, and phase.
- Save, then use the power icon to enable it. Use edit to reopen it or trash to delete it.
- The AppV2 and AppV1 full Character sheets use the same underlying Actor ActiveEffect documents.

Token workflow

- Select the token, open its status-effect palette, and toggle a configured condition icon.
- The effect is stored on the Actor or synthetic token Actor according to Foundry's token-link behavior.
- For an unlinked token, verify whether the effect should belong only to that token or to the base Actor.

> **Effects start disabled when created from the Character sheet**
>
> Creating and saving an effect does not guarantee it is active. Check the power icon. A dim/inactive power icon means the effect is disabled.

Avoid these common mistakes

- Do not use data.defense.total or data.abilities.dex.total. Foundry v14 uses system.* paths.
- Do not edit a calculated total permanently on the Actor just to simulate a temporary bonus.
- Do not use an Active Effect to spend current Health or Power Points; disabling it would reverse the apparent spend.
- Do not use empty-path Item Modifier concepts such as focusPowerForce as an Active Effect key.
- Do not assume a one-round duration will be removed by AGE automation unless the effect has an AGE automation duration flag or Foundry handles that duration in the current combat.

6 Stat path reference

These are the stat links deliberately exposed or recognized by AGE 5.0.7. Paths marked final are recalculated during Actor preparation and should normally use final phase.

Purpose	Exact key	Usual type	Phase
All tests	`system.testMod`	add	final
All attacks	`system.attackMod`	add	final
All damage	`system.dmgMod`	add / formula	final
Defense	`system.defense.total`	add	final
Speed	`system.speed.total`	add, multiply, override	final
Impact armor	`system.armor.impact`	add	final
Ballistic armor	`system.armor.ballistic`	add	final
Toughness	`system.armor.toughness.total`	add	final
Armor penalty	`system.armor.penalty`	add	final
Armor strain	`system.armor.strain`	add	final
Initiative	`system.initiative`	add	final

Resource paths

Purpose	Exact key	Recommendation
Maximum Health/Fortune	`system.health.max`	add, final
Maximum Conviction	`system.conviction.max`	add, final
Maximum Power/Magic Points	`system.powerPoints.max`	add, final
Generic Resources total	`system.resources.total`	advanced; add, final
Current Health	`system.health.value`	Do not use for spending/damage
Current Power Points	`system.powerPoints.value`	Do not use for spending
Current Conviction	`system.conviction.value`	Do not use for spending

Why maximum values are safe but current values are not

A temporary amulet may reasonably add 10 maximum Health while equipped. Damage and Power Point expenditure are transactions, not temporary overlays. Use the sheet, chat-card actions, or AGE Actions for current resources.

7 Ability, maneuver, and profile paths

Ability	Total key (final)	Test-only key (initial)
Accuracy	`system.abilities.acc.total`	`system.abilities.acc.testMod`
Communication	`system.abilities.comm.total`	`system.abilities.comm.testMod`
Constitution	`system.abilities.cons.total`	`system.abilities.cons.testMod`
Cunning	`system.abilities.cunn.total`	`system.abilities.cunn.testMod`
Dexterity	`system.abilities.dex.total`	`system.abilities.dex.testMod`
Fighting	`system.abilities.fight.total`	`system.abilities.fight.testMod`
Intelligence	`system.abilities.int.total`	`system.abilities.int.testMod`
Magic	`system.abilities.magic.total`	`system.abilities.magic.testMod`
Perception	`system.abilities.per.total`	`system.abilities.per.testMod`
Strength	`system.abilities.str.total`	`system.abilities.str.testMod`
Willpower	`system.abilities.will.total`	`system.abilities.will.testMod`

Use the total key when the effect changes the Ability everywhere, including derived calculations. Use testMod when it changes only tests and should not alter derived values such as Defense, Speed, or Power Force.

> **Rule-profile availability**
>
> Cunning and Magic are used by Dragon AGE profiles; Intelligence appears in Fantasy/Modern profiles. An effect path can exist in stored compatibility data even when the current sheet hides that Ability. Test the effect under the intended Rule Profile.

Maneuver paths

Purpose	Exact key	Usual setup
Aim modifier	`system.aim.mod`	add, final
Defend defense bonus	`system.defend.defenseBonus`	add, initial
Guard Up defense bonus	`system.guardUp.defenseBonus`	add, initial
All-Out Attack damage	`system.allOutAttack.dmgBonus`	add/formula, final
Prepared Defend modifier	`system.defend.mod`	advanced, final
Prepared Guard Up modifier	`system.guardUp.mod`	advanced, final
Prepared All-Out modifier	`system.allOutAttack.mod`	advanced, final

8 Ready-made effect recipes

Effect	Change	Expected result
Defensive Ward	`system.defense.total | add | 2 | final`	Defense is 2 higher while active.
Slowed	`system.speed.total | multiply | 0.5 | final`	Speed is halved while active.
Rooted	`system.speed.total | override | 0 | final`	Speed becomes 0.
Battle Blessing	`system.attackMod | add | 1 | final`	All attacks gain +1.
Shaken	`system.testMod | add | -1 | final`	All tests take -1.
Keen Eyes	`system.abilities.per.testMod | add | 2 | initial`	Perception tests gain +2 without changing derived stats.
Mighty	`system.abilities.str.total | add | 1 | final`	Strength and values derived from it increase.
Arcane Reserve	`system.powerPoints.max | add | 5 | final`	Maximum Power Points increase by 5.
Heavy Burden	`system.armor.strain | add | 1 | final`	Armor strain increases by 1.

Stacking example

If two active effects each add 1 to Defense, the prepared result normally gains +2. If one effect overrides Defense, its priority and Foundry's processing order become important. Prefer additive effects unless the rule genuinely sets an exact value.

Multi-change example: Wounded-style penalty

```
Change 1
key: system.speed.total
type: multiply
value: 0.5
phase: final

Change 2
key: system.testMod
type: add
value: -2
phase: final
```

Test every recipe before distributing it

Apply the effect to a disposable Actor, record the before value, confirm the exact after value, reload the world, then disable and remove the effect. Repeat under each Rule Profile the effect is intended to support.

9 AGE automation and timed effects

AGE 5.0.7 can create and remove v14-normalized Active Effects through its automation services. Automation records the effect's source and optional expiry timing in flags.age-system.automation.

Timing ID	Meaning
startOfTurn	Ready for expiry at the start of a turn
endOfTurn	Ready for expiry at the end of a turn
endOfRound	Ready for expiry at round end
endOfEncounter	Ready for expiry when the encounter ends
nextAttack	Ready after the Actor's next attack
nextTest	Ready after the Actor's next test

Lifecycle behavior

- Reminder mode posts the effect names and an Expire Now control instead of silently changing the Actor.
- Automatic mode may disable explicitly timed effects transactionally.
- Expiry disables the effect rather than deleting it, preserving an audit trail and allowing inspection.
- Ordinary hand-created effects without AGE automation duration flags are not automatically claimed by this lifecycle service.

Direct console example

```
const actor = canvas.tokens.controlled[0]?.actor;
const conditions = game.ageSystem.automation.conditions;

await conditions.applyEffect(actor, {
name: "Defensive Ward",
changes: [{
key: "system.defense.total",
type: "add",
value: "2",
phase: "final"
}],
duration: conditions.duration("endOfTurn")
});
```

Permissions still apply

A user must be allowed to modify the target Actor. AGE Actions can request permission-checked execution from the active GM; direct console code does not bypass ownership.

10 Troubleshooting

Symptom	Likely cause	Fix
Icon appears but no stat changes	The condition is descriptive only, the key is wrong, or the phase is wrong.	Inspect the effect's Changes tab and use a system.* key from this guide.
Effect saved but still does nothing	It was created disabled.	Enable it with the power icon.
Bonus appears twice	Two effects stack, or the same rule is also configured as an Item Modifier.	Inspect all Actor effects and equipped Item modifiers.
Value changes after reload	The effect targeted a calculated field at initial phase or a hidden compatibility field was involved.	Use final phase for recognized totals and retest.
Condition missing from token palette	The world is using another condition set, or Is Condition is not enabled.	Open Conditions Workshop, select Custom, mark Is Condition, Save & Close.
Workshop refuses to save	Two custom conditions use the same image.	Assign a unique icon to every custom condition.
Old imported effect looks wrong	It uses icon, numeric mode, or data.* paths from an older Foundry version.	Run the AGE migration preview/migration workflow and replace data.* with system.*.
Current HP/PP behaves strangely	A temporary Active Effect was used for resource spending.	Remove that change and use AGE damage/healing/resource actions.
Player cannot add/remove the effect	The player lacks Actor ownership or the target is controlled by a synthetic token.	Review ownership and token-link settings; use GM delegation where appropriate.

Minimal diagnostic sequence

- Use one Actor with no equipment or other effects.
- Record the raw/base sheet value.
- Create one additive effect using a known path such as system.defense.total.
- Enable it and confirm the exact delta.
- Reload Foundry and confirm persistence.
- Disable, then delete it, confirming the base value returns after each action.
- Export Operations & Diagnostics if the result differs from expectation.

11 One-page cheat sheet

Safest pattern

Use system.* + add + a numeric value. Choose final for calculated totals and initial for input modifiers. Create the effect disabled, inspect it, enable it, reload-test it, and verify that disabling it restores the original value.

Want to change...	Use this key	Type / phase
All tests	`system.testMod`	add / final
All attacks	`system.attackMod`	add / final
Defense	`system.defense.total`	add / final
Speed	`system.speed.total`	add or multiply / final
Impact armor	`system.armor.impact`	add / final
Ballistic armor	`system.armor.ballistic`	add / final
Toughness	`system.armor.toughness.total`	add / final
Maximum Health	`system.health.max`	add / final
Maximum Power Points	`system.powerPoints.max`	add / final
One Ability everywhere	`system.abilities.KEY.total`	add / final
One Ability's tests only	`system.abilities.KEY.testMod`	add / initial
Aim	`system.aim.mod`	add / final
Armor strain	`system.armor.strain`	add / final

Replace KEY with acc, comm, cons, cunn, dex, fight, int, magic, per, str, or will.

DO	DO NOT
Use system.* paths Test on a disposable Actor Use final for calculated totals Export custom conditions Keep Item Modifiers on Items	Use legacy data.* paths Spend HP/PP with an effect Assume every icon is mechanical Mix Item Modifier names into effect keys Ignore the disabled power icon

A Advanced API examples and implementation references

Apply a configured condition

```
const actor = canvas.tokens.controlled[0]?.actor;
await game.ageSystem.automation.conditions.applyStatus(
actor,
"hindered"
);
```

Remove that configured condition

```
const conditions = game.ageSystem.automation.conditions;
const effect = conditions.statusEffect(actor, "hindered");
if (effect) await conditions.removeEffect(actor, effect.uuid);
```

Create and execute an AGE Action

```
const api = game.ageSystem.api;
const action = api.actions.create("effect.apply", {
actorUuid: actor.uuid,
payload: {
effect: {
name: "Defensive Ward",
img: "icons/svg/shield.svg",
changes: [{
key: "system.defense.total",
type: "add", value: "2", phase: "final"
}]
}
}
});
console.log(await api.actions.preview(action));
await api.actions.execute(action);
```

> **Module-author rule**
>
> Feature-detect game.ageSystem.api and the actions capability. Send only UUIDs and serializable data. Do not place live Foundry Documents, DOM events, functions, or trusted user IDs inside an action payload.

Implementation references

File	What it defines
`modules/config.js`	Recognized effect keys, derived/final paths, built-in conditions
`modules/compat.js`	v14 status records and legacy effect normalization
`modules/conditions-workshop.js`	Custom condition editor, import/export, phase/type normalization
`modules/automation/condition-lifecycle.js`	Status application, timed effects, Dying assistance
`modules/automation/core-actions.js`	effect.apply and effect.remove AGE Actions
`modules/migration.js`	Legacy icon/mode/data-path migration
`modules/sheets/ageActiveEffectConfig.js`	Foundry v14 native Active Effect configuration

B Complete safe Active Effect key catalog

This appendix is the authoritative 5.0.7 list for user-authored Actor Active Effects. It is the union of the system's declared derived-data keys, declared Ability-total keys, and recognized initial-phase input modifiers. Safe means that AGE deliberately recognizes the path and phase shown; a path can still be irrelevant to an Actor type or hidden by the selected Rule Profile.

> **Use only the keys in this appendix unless you are developing and testing system code**
>
> The list intentionally excludes current Health, Power Points, and Conviction; legacy data.* paths; commented-out internal paths; and Item Modifier concepts whose path mask is empty. Start with add and a numeric value, then use multiply or override only when the rule genuinely requires it.

Final-phase general and derived keys

Purpose	Exact key	Usual type
Armor penalty	`system.armor.penalty`	add
Armor strain	`system.armor.strain`	add
All damage	`system.dmgMod`	add; numeric or formula value
All tests	`system.testMod`	add
All attacks	`system.attackMod`	add
Defense	`system.defense.total`	add
Impact armor	`system.armor.impact`	add
Ballistic armor	`system.armor.ballistic`	add
Toughness	`system.armor.toughness.total`	add
Speed	`system.speed.total`	add, multiply, or override
Prepared Defend modifier	`system.defend.mod`	add; advanced/profile-dependent
Prepared Guard Up modifier	`system.guardUp.mod`	add; advanced/profile-dependent
Prepared All-Out modifier	`system.allOutAttack.mod`	add; advanced/profile-dependent
All-Out Attack damage	`system.allOutAttack.dmgBonus`	add; numeric or formula value
Maximum Health/Fortune	`system.health.max`	add
Maximum Conviction	`system.conviction.max`	add
Maximum Power/Magic Points	`system.powerPoints.max`	add
Aim modifier	`system.aim.mod`	add
Initiative	`system.initiative`	add; profile-dependent
Generic resource total	`system.resources.total`	add; advanced/profile-dependent

Every key in the table above uses final phase.

B.1 Ability totals - final phase

Use these paths when the Ability itself changes and AGE should recalculate values that depend on it. The usual type is add. Cunning and Magic belong to Dragon AGE profiles; Intelligence belongs to Fantasy/Modern AGE profiles.

Ability	Exact key	Type / phase
Accuracy	`system.abilities.acc.total`	add / final
Communication	`system.abilities.comm.total`	add / final
Constitution	`system.abilities.cons.total`	add / final
Cunning	`system.abilities.cunn.total`	add / final
Dexterity	`system.abilities.dex.total`	add / final
Fighting	`system.abilities.fight.total`	add / final
Intelligence	`system.abilities.int.total`	add / final
Magic	`system.abilities.magic.total`	add / final
Perception	`system.abilities.per.total`	add / final
Strength	`system.abilities.str.total`	add / final
Willpower	`system.abilities.will.total`	add / final

Test-only and maneuver inputs - initial phase

Purpose	Exact key	Type / phase
Accuracy tests	`system.abilities.acc.testMod`	add / initial
Communication tests	`system.abilities.comm.testMod`	add / initial
Constitution tests	`system.abilities.cons.testMod`	add / initial
Cunning tests	`system.abilities.cunn.testMod`	add / initial
Dexterity tests	`system.abilities.dex.testMod`	add / initial
Fighting tests	`system.abilities.fight.testMod`	add / initial
Intelligence tests	`system.abilities.int.testMod`	add / initial
Magic tests	`system.abilities.magic.testMod`	add / initial
Perception tests	`system.abilities.per.testMod`	add / initial
Strength tests	`system.abilities.str.testMod`	add / initial
Willpower tests	`system.abilities.will.testMod`	add / initial
Defend defense bonus	`system.defend.defenseBonus`	add / initial
Guard Up defense bonus	`system.guardUp.defenseBonus`	add / initial

Catalog total: 44 safe keys

There are 20 final-phase general/derived keys, 11 final-phase Ability totals, and 13 initial-phase input modifiers. Before distributing an effect, test enable, reload, disable, and delete behavior on a disposable Actor under every intended Rule Profile.

This guide explains behavior implemented by AGE System 5.0.7 and does not reproduce complete rules text from any AGE game line. The GM remains responsible for selecting conditions and numerical changes that match the campaign's rules.